



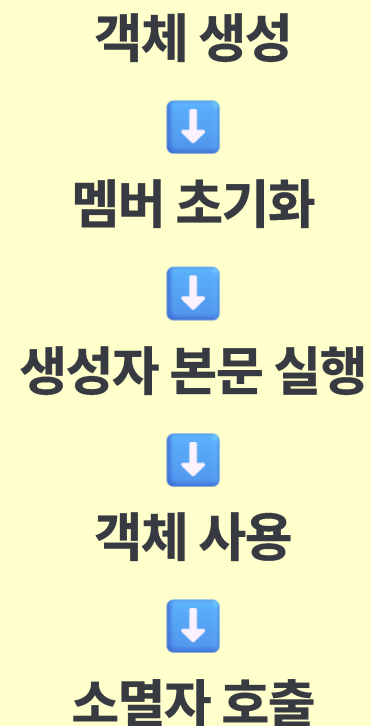
04. 생성자와 소멸자, 초기화

Heesung Oh

<https://github.com/3dapi>



- 생성자로 객체의 초기 상태 구성
- 소멸자로 객체 수명 종료 시 자원 정리
- 기본 초기화, 값 초기화, 직접 초기화, 복사 초기화, 리스트 초기화
- 멤버 초기화 리스트와 초기화 순서
- 기본 멤버 초기값, 위임 생성자, explicit
- = default, = delete, std::initializer_list
- const 정확성과 클래스 인터페이스
- constexpr, consteval, constexpr





- 객체가 생성될 때 호출되는 특별한 멤버 함수
- 생성자 이름은 클래스 이름과 같음
- 반환 타입을 작성하지 않음
- 객체의 초기 상태를 만드는 역할

```
struct Point { int x, y; };
```

```
class Rectangle
```

```
{  
public:  
    Rectangle()  
    {  
        position.x = 0;  
        position.y = 0;  
        width = 1;  
        height = 1;  
    }  
}
```

```
private:  
    Point position;  
    int width;  
    int height;  
};
```

```
Rectangle rectangle; //Rectangle() 생성자 호출
```



- 멤버 변수의 초기값 설정
- 전달받은 값의 유효성 검사
- 객체 타입 멤버의 생성
- 객체가 사용할 자원의 준비
- 객체의 상태 규칙 설정

- 생성자는 객체의 모든 기능을 처리하는 함수가 아님
- 생성과 초기 상태 구성에 필요한 작업에 집중

```
class Circle
{
public:
    Circle(int x, int y, int radius)
    {
        center.x = x;
        center.y = y;

        if (radius > 0)
        {
            this->radius = radius;
        }
        else
        {
            this->radius = 1;
        }
    }

private:
    Point center;
    int radius;
};
```



4.1.2 기본 생성자 (default constructor)

- 인수 없이 호출할 수 있는 생성자
- 객체를 인수 없이 선언하면 기본 생성자 호출
- 기본 생성자 호출을 위해 빈 괄호를 붙이지 않음

```
class Rectangle
```

```
{  
public:  
    Rectangle()  
    {  
        position.x = 0;  
        position.y = 0;  
        width = 1;  
        height = 1;  
    }  
}
```

```
private:  
    Point position;  
    int width;  
    int height;  
};
```

```
Rectangle first;  
Rectangle second{};
```

- 중괄호는 객체 생성 의도가 분명함
- 빈 괄호는 매개변수가 없는 함수를 선언하는 형태로 해석될 수 있음

```
Rectangle rectangle(); // 객체 선언이 아니라 함수 선언
```



- 클래스에 생성자가 하나도 없으면 컴파일러가 암시적으로 기본 생성자 선언
- 컴파일러 제공 기본 생성자는 기본형 멤버를 초기화 안함

```
class Rectangle
```

```
{  
private:  
    Point position;  
    int width;  
    int height;  
};
```

```
Rectangle rectangle;
```

- 기본 초기화에서는 width, height에 정해지지 않은 값이 남을 수 있음
- 사용자가 생성자를 하나라도 선언하면 기본 생성자가 자동으로 제공되지 않을 수 있음

```
class Rectangle
```

```
{  
public:  
    Rectangle(int x, int y, int width, int height);  
  
private:  
    Point position;  
    int width;  
    int height;  
};
```

```
// Rectangle first; // 오류
```

```
Rectangle second(10, 20, 100, 50);
```



4.1.4 매개변수가 있는 생성자

- 객체마다 서로 다른 초기 상태가 필요할 때 사용
- 생성자의 매개변수로 객체 생성에 필요한 값을 인터페이스로 표현
- 매개 변수만 있는 생성자로 멤버 변수의 필요한 초기값 설정

```
class Rectangle
{
public:
    Rectangle(int x, int y, int width, int height)
    {
        position.x = x;
        position.y = y;

        this->width = width > 0 ? width : 1;
        this->height = height > 0 ? height : 1;
    }

private:
    Point position;
    int width;
    int height;
};

Rectangle first(10, 20, 100, 50);
Rectangle second(0, 0, 640, 480);
// Rectangle invalidRectangle; // 오류
```



- 생성자도 매개변수의 개수나 타입이 다르면 오버로딩 가능
- 객체 생성에 사용한 인수에 맞는 생성자 선택
- 각 생성 방법의 목적이 분명해야 함

```
class Rectangle
{
public:
    Rectangle()
    {
        position = {0, 0}; width = 1; height = 1;
    }
    Rectangle(int width, int height)
    {
        position = {0, 0};
        this->width = width;
        this->height = height;
    }
    Rectangle(int x, int y, int width, int height)
    {
        position = {x, y};
        this->width = width;
        this->height = height;
    }

private:
    Point position;
    int width;
    int height; };
```

```
Rectangle first;
Rectangle second(100, 50);
Rectangle third(10, 20, 100, 50);
```



4.1.5 생성자 오버로딩 주의점

구분	성립 여부	설명
매개변수 개수	성립	개수가 다르면 서로 다른 생성자
매개변수 타입	성립	타입이 다르면 서로 다른 생성자
매개변수 타입 순서	성립	타입 배치가 다르면 구분 가능
반환 타입	해당 없음	생성자에는 반환 타입이 없음
매개변수 이름	성립하지 않음	이름은 함수 구분 기준이 아님

```
class Sample
{
public:
    Sample();
    Sample(int value = 0);
};
```

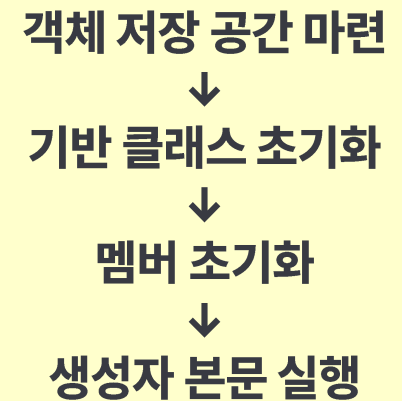
// Sample sample; // 오류: 호출할 생성자가 모호함

- 기본 인수와 오버로딩을 함께 사용할 때 호출 모호성 확인
- 중복되는 초기화 코드는 기본 멤버 초기값과 위임 생성자로 줄일 수 있음



4.1.7 객체 생성과 초기화 순서

- 객체 생성은 생성자 본문만 실행하는 과정이 아님
- 저장 공간 마련
- 기반 클래스와 멤버 초기화
- 생성자 본문 실행



```
class Rectangle
{
public:
    Rectangle(int x, int y, int width, int height)
    {
        position.x = x;
        position.y = y;
        this->width = width;
        this->height = height;
    }

private:
    Point position;
    int width;
    int height;
};
```

- 생성자 본문이 실행될 때 멤버는 이미 초기화 과정을 거친 상태
- 본문 안의 코드는 초기화가 아니라 대입



- 객체의 수명이 끝날 때 호출되는 특별한 멤버 함수
- 클래스 이름 앞에 ~를 붙여 작성
- 반환 타입 없음
- 매개변수 없음
- 클래스마다 하나만 선언 가능
- 오버로딩 불가

```
class Rectangle
{
public:
    Rectangle();
    ~Rectangle();

private:
    Point position;
    int width;
    int height;
};
```

소멸자의 역할

- 동적으로 할당한 메모리 해제
- 열린 파일 닫기
- 잠금 해제
- 콜백이나 연결 해제
- 운영체제 자원 반환



4.2.2 객체의 소멸 시점

객체 종류	소멸 시점
지역 객체	선언된 블록을 벗어날 때
전역 객체	프로그램 종료 시
정적 객체	프로그램 종료 시
동적 객체	delete 실행 시
동적 객체 배열	delete[] 실행 시

```
class Trace
{
public:
    Trace(const std::string& name) : name(name)
    {
        std::cout << name << " 생성\n";
    }
    ~Trace()
    {
        std::cout << name << " 소멸\n";
    }
private:
    std::string name;
};
```

```
int main()
{
    Trace trace("local");
    std::cout << "main 실행 중\n";
}

local 생성
main 실행 중
local 소멸
```





- **new** 로 생성한 객체는 **delete** 로 소멸
- **new []** 로 생성한 객체 배열은 **delete []** 로 소멸
- **delete** 는 소멸자를 호출한 뒤 메모리를 해제
- **delete []** 는 배열 요소의 소멸자를 역순으로 호출

```
Trace* trace = new Trace("dynamic");  
delete trace;  
trace = nullptr;
```

```
class Sample  
{  
public:  
    Sample() {  
        std::cout << "생성: " << this << '\n';  
    }  
    ~Sample() {  
        std::cout << "소멸: " << this << '\n';  
    }  
};
```

```
Sample* samples = new Sample[3];  
delete[] samples;
```

- **new[]와 delete를 섞으면 미정의 동작**



- 여러 객체는 생성된 순서의 반대로 소멸
- 배열 요소도 앞에서부터 생성되고 뒤에서부터 소멸
- 멤버 객체는 바깥 객체보다 먼저 생성되고 나중에 소멸

```
class Border
{
public:
    Border() {
        std::cout << "Border 생성\n";
    }
    ~Border() {
        std::cout << "Border 소멸\n";
    }
};

class Window
{
public:
    Window() {
        std::cout << "Window 생성\n";
    }
    ~Window() {
        std::cout << "Window 소멸\n";
    }
private:
    Border border;
};
```

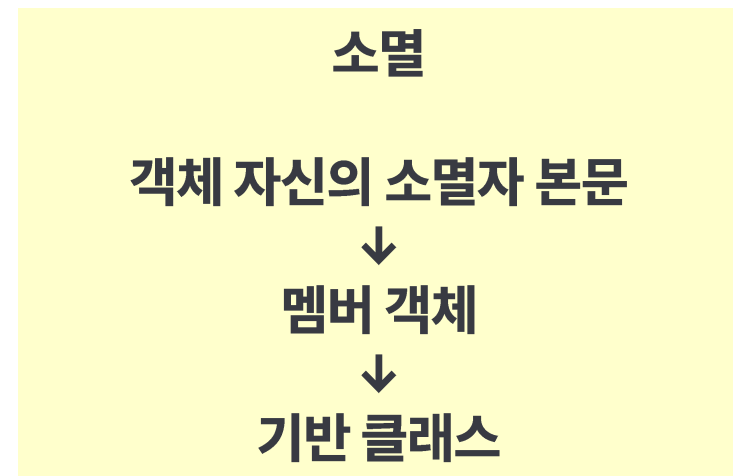
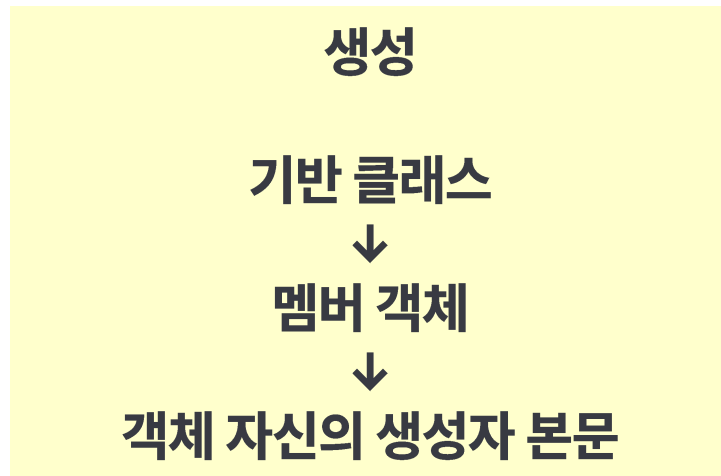
Border 생성
Window 생성
Window 소멸
Border 소멸





4.2.5 구성 요소의 생성과 소멸 순서

구성 요소	생성 순서	소멸 순서
기본 클래스	가장 먼저 생성	가장 나중에 소멸
멤버 객체	선언된 순서대로 생성	선언된 역순으로 소멸
객체 자신의 생성자·소멸자 본문	생성 과정의 마지막에 실행	소멸 과정의 처음에 실행



- 객체를 구성하는 내부 요소를 먼저 준비 → 바깥 객체의 생성자 본문 실행 가능
- 정리 과정에서는 바깥 객체의 소멸자 본문이 먼저 실행



- 소멸자는 예외를 외부로 전달하지 않는 것이 원칙
- 실패 가능성이 낮고 확실하게 정리할 수 있는 작업에 적합
- 복잡한 저장 작업이나 사용자 확인이 필요한 작업 → 소멸자에 몰아넣지 않음
- 소멸자는 일반적으로 직접 호출하지 않음

Rectangle rectangle;

// rectangle.~Rectangle(); // 직접 호출하지 않음

직접 호출의 문제

- 실제 객체 수명은 끝나지 않았는데 소멸자 본문만 실행될 수 있음
- 이후 객체가 다시 소멸할 때 같은 자원을 두 번 해제할 수 있음
- 소멸은 객체 수명 규칙에 맡기는 것이 기본
- 새 코드에서는 직접 new, delete 관리보다 컨테이너와 스마트 포인터 우선





- 초기화는 객체가 처음 만들어질 때 값을 설정하는 과정
- 대입은 이미 존재하는 객체의 값을 변경하는 연산
- 문법 형태에 따라 초기화 방식과 생성자 선택 규칙이 달라질 수 있음

```
int first = 10; // 초기화  
int second;  
second = 10; // 대입
```

초기화 방식	문법	주요 동작
기본 초기화	<code>T object;</code>	클래스는 기본 생성자 호출, 기본형 지역 변수는 값이 정해지지 않을 수 있음
값 초기화	<code>T object{};</code>	기본형과 집합체 멤버를 0에 해당하는 값으로 초기화
직접 초기화	<code>T object(value);</code>	인수에 맞는 생성자 직접 선택
복사 초기화	<code>T object = value;</code>	암시적 변환 허용
리스트 초기화	<code>T object{value};</code>	축소 변환 제한, 집합 초기화 가능



- 초기값 없이 객체를 선언하면 기본 초기화 적용
- 클래스 타입에는 기본 생성자 호출
- 기본형 지역 변수에는 정해지지 않은 값이 남을 수 있음

```
int count;  
double ratio;  
Rectangle rectangle;
```

```
struct Point  
{  
    int x;  
    int y;  
};
```

```
Point point;
```

- count, ratio, point.x, point.y를 초기화 없이 읽으면 올바른 결과를 기대할 수 없음
- 기본 생성자가 있어도 기본형 멤버를 0으로 만든다는 뜻은 아님

T object;



클래스 타입: 기본 생성자 호출

기본형 지역 변수: 값이 정해지지 않을 수 있음



4.3.2 값 초기화

- 빈 괄호나 빈 중괄호를 사용하면 값 초기화 적용 가능
- 기본형은 0에 해당하는 값으로 초기화
- 집합체의 기본형 멤버도 0에 해당하는 값으로 초기화
- 클래스 타입에 기본 생성자가 있으면 기본 생성자 호출

```
int count{};  
double ratio{};  
Point point{};  
Rectangle rectangle{};
```

```
count = 0  
ratio = 0.0  
point.x = 0  
point.y = 0
```

동적 객체의 값 초기화

```
int* first = new int; // 초기값 없음  
int* second = new int(); // 0으로 초기화
```

```
delete first;  
delete second;
```

- 동적 객체도 초기화 방식에 따라 결과가 달라짐



4.3.3 직접 초기화와 복사 초기화

직접 초기화

```
Rectangle first(10, 20, 100, 50);  
Rectangle second{10, 20, 100, 50};
```

- 인수에 맞는 생성자를 직접 선택
- explicit 생성자 호출 가능

복사 초기화

```
int value = 10;  
Rectangle rectangle = Rectangle(10, 20, 100, 50);
```

- 등호가 있지만 이미 존재하는 객체에 값을 대입하는 것이 아님
- 새로운 객체를 생성하는 초기화 문법

```
class Length
```

```
{  
public:  
    Length(int value)  
        : value(value)  
    {  
    }  
}
```

```
private:  
    int value;  
};
```

```
Length length = 100;
```

- explicit이 없으면 변환 생성자가 복사 초기화에 사용될 수 있음



- 기본형, 구조체, 배열과 클래스 객체에 공통된 형태로 사용
- 축소 변환을 허용하지 않음
- 빈 중괄호는 기본값을 사용한 초기화에 적합

```
int value{10};  
Point point{10, 20};  
Rectangle rectangle{10, 20, 100, 50};
```

```
int first = 3.14;  
int second(3.14);
```

```
// int third{3.14}; // 오류
```

```
double value = 3.14;
```

```
// int number{value}; // 오류
```

```
int count{};  
Point position{};  
Rectangle rectangle{}
```

- 잘못된 형 변환을 컴파일 시점에 찾는 데 도움



- 집합체는 멤버를 선언된 순서대로 직접 초기화할 수 있는 타입
- 생성자를 직접 선언하지 않고
공개 멤버만 가지는 단순한 구조체에 적합
- 배열도 집합 초기화 가능
- 일부 값만 작성하면 나머지 요소는 0으로 초기화

```
struct Point { int x, y; };
```

```
Point first{10, 20};  
Point second{30, 40};
```

```
int values[5]{1, 2};
```

```
struct Size  
{  
    int width;  
    int height;  
};
```

```
struct Area
```

```
{  
    Point position;  
    Size size;  
};
```

```
Area area{{10, 20}, {100, 50}};
```

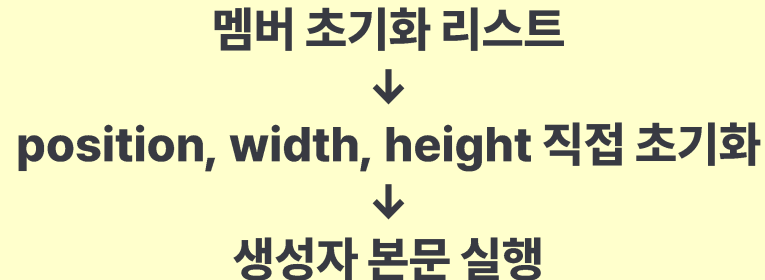


- 생성자 본문이 실행되기 전에 멤버를 직접 초기화
- 생성자의 매개변수 목록 뒤에 콜론 작성
- 초기화할 멤버 나열

```
class Rectangle
{
public:
    Rectangle(Point position, int width, int height)
        : position(position), width(width),
          height(height)
    {
    }

private:
    Point position;
    int width;
    int height;
};

Rectangle rectangle({10, 20}, 100, 50);
```



- 생성자 본문이 실행될 때는 모든 멤버 초기화가 끝난 상태



● 생성자 본문의 대입

```
class Label
{
public:
    Label(const std::string& text)
    {
        this->text = text;
    }

private:
    std::string text;
};
```

- text 멤버가 먼저 기본 생성됨
- 생성자 본문에서 전달받은 문자열이 대입됨

● 멤버 초기화 리스트

```
class Label
{
public:
    Label(const std::string& text)
        : text(text)
    {
    }

private:
    std::string text;
};
```

- 전달받은 값으로 text를 직접 초기화
- 특별한 이유가 없다면 생성자에서는 멤버 초기화 리스트 우선



4.4.2 멤버 초기화 순서

- 멤버는 초기화 리스트에 작성한 순서가 아니라 클래스에 선언된 순서대로 초기화
- 초기화 리스트도 멤버 선언 순서와 같게 작성

class Rectangle

```
{  
public:  
    Rectangle(int width, int height)  
        : height(height), width(width){}  
private:  
    int width; int height;  
};
```

- 실제 초기화 순서: width → height
- 초기화 리스트 작성 순서: height → width

class Size

```
{  
public:  
    Size(int width)  
        : width(width), area(width * width){}  
private:  
    int width; int area;  
};
```

- 한 멤버의 초기값이 다른 멤버에 의존하면 선언 순서가 특히 중요



4.4.3 객체 타입 멤버 초기화

- 클래스가 객체 타입 멤버를 가지면 그 멤버의 생성자가 먼저 호출
- 객체 타입 멤버에 기본 생성자가 없으면
멤버 초기화 리스트에서 반드시 초기화해야 함

```
struct Point
```

```
{  
    Point(int x, int y)  
        : x(x), y(y) {}  
    int x, y;  
};
```

```
class Rectangle
```

```
{  
public:  
    Rectangle(int x, int y, int width, int height)  
        : position(x, y), width(width), height(height)  
    {}  
private:  
    Point position;  
    int width;  
    int height;  
};
```

- position은 Point(x, y) 생성자로 초기화
- 생성자 본문에서 대입하는 방식은 사용할 수 없음



4.4.4 const 멤버와 참조 멤버 초기화

```
// const 멤버  
class Shapeld  
{  
public:  
    Shapeld(int id)  
        : id(id){}  
  
private:  
    const int id;  
};
```

- **const** 멤버는 객체가 생성될 때 반드시 초기화
- 생성자 본문에서 대입 불가

```
// 참조 멤버  
class PointView  
{  
public:  
    PointView(Point& point)  
        : point(point)  
    {  
    }  
  
private:  
    Point& point;  
};
```

- 참조 멤버는 생성 시점에 참조 대상을 결정해야
- 참조하는 객체의 수명을 함께 고려해야



기능	목적
기본 멤버 초기값	여러 생성자에서 공통으로 사용할 초기값
위임 생성자	다른 생성자에 초기화 작업 위임
explicit	생성자를 통한 암시적 변환 제한
= default	컴파일러의 기본 동작을 명시적으로 요청
= delete	특정 생성, 변환, 복사, 대입 금지
std::initializer_list	중괄호 안의 여러 값 전달

- C++11 이후 객체 생성 방법을 더 분명하게 표현할 수 있음
- 생성자 중복을 줄이고 의도하지 않은 생성 방법을 막을 수 있음





- 멤버를 선언하면서 기본값 지정
- 생성자가 해당 멤버를 별도로 초기화하지 않으면 기본 멤버 초기값 사용
- 여러 생성자에서 공통 초기값을 한곳에 표현

```
class Rectangle
{
public:
    Rectangle() = default;

    Rectangle(int width, int height)
        : width(width), height(height)
    {
    }

    Rectangle(Point position, int width, int height)
        : position(position), width(width),
height(height)
    {
    }

private:
    Point position{0, 0};
    int width = 1;
    int height = 1;
};
```

```
Rectangle first;
Rectangle second(100, 50);
Rectangle third({10, 20}, 100, 50);
```

- 멤버 초기화 리스트에서 지정한 값이 기본 멤버 초기값보다 우선



- 한 생성자가 같은 클래스의 다른 생성자를 호출
- 공통 초기화와 유효성 검사를 하나의 생성자에 모을 수 있음
- 위임 생성자는 같은 클래스의 생성자 하나만 호출

```
class Rectangle
{
public:
    Rectangle()
        : Rectangle({0, 0}, 1, 1)
    {
    }

    Rectangle(int width, int height)
        : Rectangle({0, 0}, width, height)
    {
    }
}
```

```
Rectangle(Point position, int width, int height)
: position(position), width(width), height(height)
{
}
```

- 위임 생성자에서는 다른 생성자 호출과 멤버 초기화를 함께 작성할 수 없음



- explicit은 생성자를 통한 암시적 변환을 제한
- 직접 초기화는 허용
- 복사 초기화와 암시적 함수 인수 변환은 제한

```
class Length
{
public:
    explicit Length(int value)
        : value(value)
    {
    }

private:
    int value;
};
```

```
void PrintLength(const Length& length)
{
    std::cout << length.Value() << '\n';
}
```

```
Length first(100);
Length second{200};
// Length third = 300; // 오류
// PrintLength(400); // 오류

PrintLength(Length(400));
```

- 클래스 타입으로의 암시적 변환이 명확한 의미를 가지지 않으면 단일 인수 생성자에 explicit 지정



- 컴파일러가 제공하는 **기본 동작을 명시적으로** 사용
- 기본 생성자, 소멸자, 복사·이동 관련 특별 멤버 함수에 사용
- 아무 작업도 하지 않는다는 뜻이 아님
- 각 멤버의 생성과 소멸 규칙에 맞는 기본 동작을 컴파일러가 생성

```
class Rectangle
{
public:
    Rectangle() = default;
    ~Rectangle() = default;

private:
    Point position{0, 0};
    int width = 1, height = 1;
};
```

```
class Point
{
public:
    Point() = default;

    Point(int x, int y)
        : x(x), y(y) {}

private:
    int x = 0, y = 0;
};
```

- 다른 생성자를 선언한 뒤에도 기본 생성자를 명시적으로 제공 가능



- 특정 함수를 사용할 수 없도록 명시
- 허용하지 않을 생성, 변환, 복사, 대입을 컴파일 시점에 차단

```
class Rectangle
{
public:
    Rectangle() = delete;

    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height)
    {
    }
private:
    Point position;
    int width, height;
};

// Rectangle first; // 오류
Rectangle second({10, 20}, 100, 50);
```

```
class Length
{
public:
    explicit Length(int value)
        : value(value)
    {
    }

    Length(double value) = delete;
private:
    int value;
};

// Length invalid(3.14); // 오류
```



4.5.6 std::initializer_list

- 중괄호 안의 여러 값을 생성자에 전달
- std::initializer_list<T>를 매개변수로 받는 생성자 사용
- 부족한 값이나 초과한 값의 처리 규칙을 클래스가 결정

```
#include <initializer_list>

class Matrix2x2
{
public:
    Matrix2x2(std::initializer_list<int> values)
    {
        // ...
    }
private:
    int elements[4];
};

Matrix2x2 first{1, 2, 3, 4};
Matrix2x2 second{1, 2};
```





- 객체를 변경하는 기능과 변경하지 않는 기능을 타입과 인터페이스에 반영
- 조회 함수는 상수 멤버 함수로 선언
- 읽기만 하는 객체는 상수 참조로 전달
- 내부 상태 반환할 때 호출 객체 상수성 유지

```
class Rectangle
{
public:
    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height)
    { }
    void MoveTo(Point position);
    void Resize(int width, int height);
    int Area() const;
    const Point& Position() const;
private:
    Point position;
    int width;
    int height;
};
```

함수	의미
MoveTo(), Resize()	객체 상태 변경
Area() const, Position() const	객체 상태 조회



4.6.1 상수 멤버 함수와 this

- 객체의 상태를 변경하지 않는 멤버 함수는 매개변수 목록 뒤에 const 지정
- 상수 멤버 함수 안에서는 일반 데이터 멤버 변경 불가
- 상태를 변경하는 일반 멤버 함수 호출 불가
- const는 반환값이 아니라 호출 객체에 적용

```
class Rectangle
{
public:
    int Area() const
    {
        // width = 100; // 오류
        return width * height;
    }

private:
    Point position;
    int width;
    int height;
};
```

멤버 함수	this가 가리키는 객체	일반 멤버 변경
일반 멤버 함수	변경 가능한 객체	가능
상수 멤버 함수	변경할 수 없는 객체	불가능

- 상수 멤버 함수는 호출 객체의 논리적 상태를 변경하지 않는다는 의도 표현



- `const`로 선언한 객체는 상태를 변경할 수 없음
- 상수 객체에서는 상수 멤버 함수만 호출 가능
- 일반 객체는 일반 멤버 함수와 상수 멤버 함수를 모두 호출 가능

```
const Rectangle rectangle({10, 20}, 100, 50);
```

```
int area = rectangle.Area();
```

```
// rectangle.MoveTo({30, 40}); // 오류
```

```
// rectangle.Resize(200, 100); // 오류
```

```
Rectangle rectangle({10, 20}, 100, 50);
```

```
rectangle.MoveTo({30, 40});
```

```
int area = rectangle.Area();
```

호출 객체	일반 멤버 함수	상수 멤버 함수
일반 객체	호출 가능	호출 가능
상수 객체	호출 불가능	호출 가능

`const`는 초기화가 끝난 객체를 해당 이름이나 참조를 통해 변경하지 못하도록 제한



4.6.3 const 오버로딩

- 멤버 함수는 매개변수 목록 뒤의 const 여부로 오버로딩 가능
- 일반 객체와 상수 객체에서 다른 접근 권한 제공

```
class Rectangle
{
public:
    Point& Position()
    {
        return position;
    }

    const Point& Position() const
    {
        return position;
    }
};
```

```
private:
    Point position;
    int width;
    int height;
};
```

```
Rectangle rectangle({10, 20}, 100, 50);
rectangle.Position().x = 30;
```

```
const Rectangle constRectangle({10, 20}, 100, 50);
int x = constRectangle.Position().x;
```

```
// constRectangle.Position().x = 30; // 오류
```

호출 객체	선택 함수	반환 타입
Rectangle	일반 멤버 함수	Point&
const Rectangle	상수 멤버 함수	const Point&



4.6.4 상수 참조 매개변수와 반환값

- 객체를 복사하지 않고 읽기 전용으로 전달하려면 **const T&** 사용
- 상수 참조를 통해서만 상수 멤버 함수만 호출 가능
- 내부 상태를 참조로 반환할 때도 상수성 유지 필요

```
void PrintArea(const Rectangle& rectangle)
{
    std::cout << rectangle.Area() << '\n';
    // rectangle.MoveTo({0, 0}); // 오류
}
```

```
const Point& Position() const
{
    return position;
}
```

반환 방식	특징	주의점
Point	독립된 값 반환	복사 또는 이동 가능
Point&	내부 객체 수정 가능	외부에서 상태 직접 변경 가능
const Point&	내부 객체 읽기 전용 참조	원본 객체 수명에 의존

- 작고 복사 비용이 낮은 타입은 값으로 반환하는 것이 단순할 수 있음



4.6.5 논리적 상수성과 mutable

- 상수 멤버 함수는 객체의 논리적 상태를 변경하지 않아야 함
- 논리적 상태는 객체가 외부에 나타내는 의미와 관찰 가능한 상태
- 조회 횟수나 캐시 같은 내부 구현 정보는 논리적 상태와 구분 가능
- **mutable** 멤버는 상수 멤버 함수에서도 변경 가능

```
class Rectangle
{
public:
    int Area() const {
        ++areaQueryCount;
        return width * height;
    }
    int AreaQueryCount() const {
        return areaQueryCount;
    }

private:
    Point position;
    int width;
    int height;
    mutable int areaQueryCount = 0;
};
```

구분	의미
물리적 상수성	객체의 저장된 데이터 멤버를 변경하지 않음
논리적 상수성	외부에 나타나는 의미와 상태를 변경하지 않음



4.7 constexpr와 컴파일 타임 초기화

- 프로그램의 일부 값은 컴파일 시점에 결정 가능
- 컴파일 시점에 계산한 값은 배열 크기, 템플릿 인수, 정적 검사에 사용 가능
- **constexpr**: 변수나 함수가 상수 표현식에 참여할 수 있음을 표현

```
constexpr int width = 100;  
constexpr int height = 50;  
constexpr int area = width * height;
```

```
int ReadWidth();
```

```
const int width = ReadWidth();  
// constexpr int area = width * 50; // 오류
```

구분	의미
const	객체를 통한 변경 제한
constexpr	상수 표현식에서 사용할 수 있음

- **const 객체** 라고 항상 컴파일 타임 상수인 것은 아님
- **constexpr 변수**는 상수 표현식으로 초기화되어야 함



4.7.3 constexpr 함수

- 상수 표현식 안에서 호출될 수 있는 함수
- 상수 인수를 전달하면 컴파일 시점에 계산 가능
- 실행 시점 값을 전달하면 일반 함수처럼 실행 가능

```
constexpr int CalculateArea(int width, int height)
{
    return width * height;
}
```

```
constexpr int area = CalculateArea(100, 50);
static_assert(CalculateArea(10, 20) == 200);
```

```
int width = 0;
int height = 0;
```

```
std::cin >> width >> height;
```

```
int area = CalculateArea(width, height);
```

- 함수가 constexpr라고 해서 모든 호출이 컴파일 시점에 평가되는 것은 아님
- 호출 위치와 인수가 상수 표현식 조건을 만족해야 컴파일 시점 평가 가능



- consteval (const evaluation: C++20)

- ◆ 상수 평가
- ◆ 모든 호출이 반드시 컴파일 시점에 평가
- ◆ 실행 시점에 결정되는 값은 전달 불가

```
constexpr int Square(int value)
{
    return value * value;
}

constexpr int result = Square(10);

// int value = 0;
// std::cin >> value;
// int result = Square(value); // 오류
```

- constinit (constant initialization: C++20)

- ◆ 상수 초기화
- ◆ 정적 저장 기간 또는 스레드 저장 기간 변수의 정적 초기화 요구
- ◆ 변수를 읽기 전용으로 만들지는 않음

```
constinit int objectCount = 0;

void IncreaseCount()
{
    ++objectCount;
}
```



4.7.7 const, constexpr, consteval, constinit

키워드	주요 적용 대상	의미	변경 가능 여부
const	변수, 객체, 멤버 함수	객체를 통한 변경을 제한	선언에 따라 변경 불가
constexpr	변수, 함수, 생성자	상수 표현식에서 사용할 수 있도록 선언	constexpr 변수는 변경 불가
consteval	함수	모든 호출을 컴파일 시점에 평가 요구	해당 없음
constinit	정적·스레드 저장 기간 변수	정적 초기화 보장	변경 가능





- 생성자는 객체가 생성될 때 호출, 멤버를 초기화, 초기 상태 구성
- 기본 생성자는 인수 없이 호출할 수 있는 생성자
- 생성자가 하나라도 있으면 기본 생성자는 자동으로 생성 안됨
- 소멸자는 객체의 수명이 끝날 때 호출되어 자원 정리
- 지역 객체는 블록을 벗어날 때,
동적 객체는 delete 실행 시 소멸
- 같은 범위의 객체는 생성 순서의 반대로 소멸





- 초기화 문법에 따라 생성자 선택과 형 변환 규칙이 달라질 수 있음
- 중괄호 초기화는 축소 변환을 제한
- 멤버 초기화 리스트는 생성자 본문 실행 전에 멤버를 직접 초기화
- 실제 멤버 초기화 순서는 멤버 선언 순서
- `const` 멤버, 참조 멤버, 기본 생성자가 없는 객체 멤버는 멤버 초기화 리스트 필요

